

Formbricks On-Premise Hosting Plan

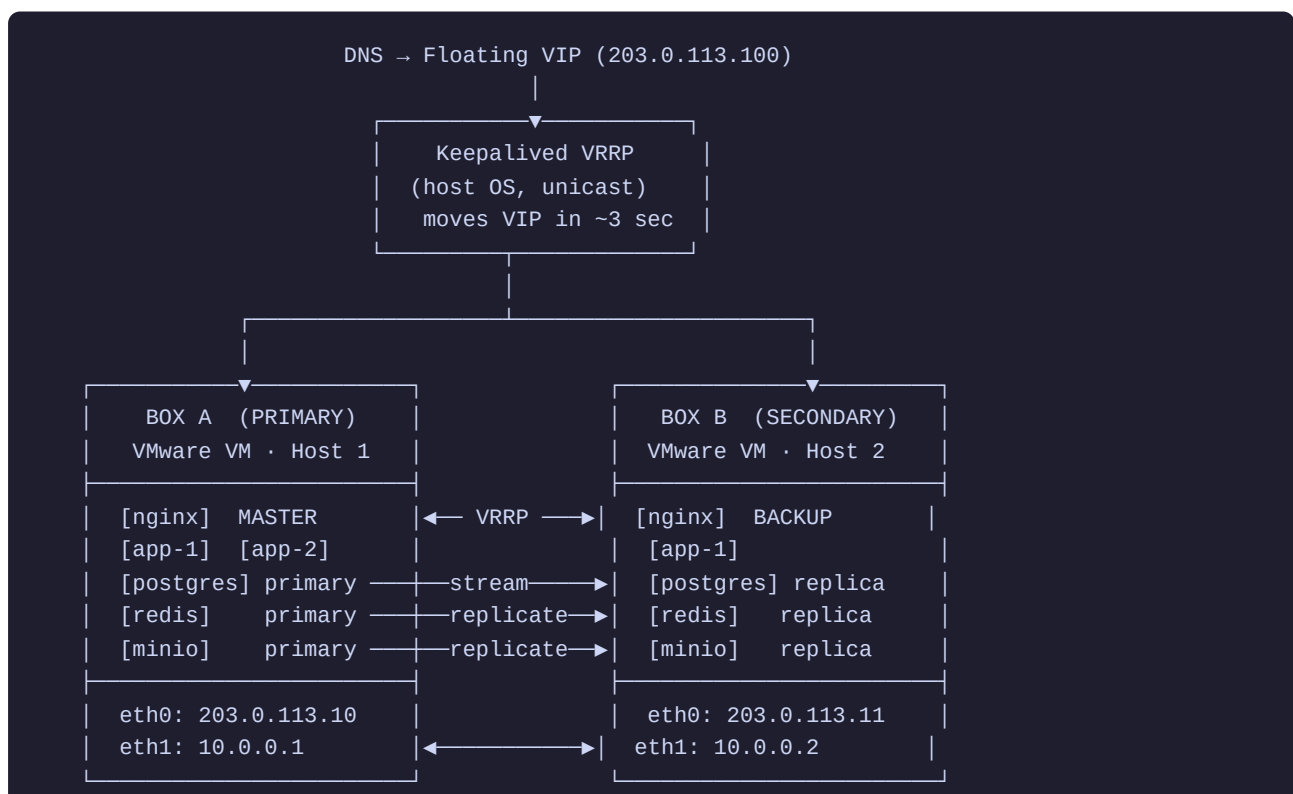
Two-VM High Availability · 5,000 Users / 30-Minute Peak Load · May 2026

1. Load Analysis

Before sizing infrastructure, the expected peak workload is translated into concrete server metrics.

Metric	Calculation	Result
Arrival rate	5,000 users / 30 min	~167 users/min
Avg session duration	Survey fill + submit	~4–8 min
Peak concurrent users	167/min × 6 min avg	~800–1,000 concurrent
Requests per user	Page load + API calls + submit	~15–25 req/session
Total requests	5,000 × 20	~100,000 requests
Average throughput	100,000 req / 1,800 sec	~55 req/sec
Peak throughput (burst)	2–3× average	~110–165 req/sec

2. Architecture Overview



16 cores · 32 GB RAM
500 GB NVMe SSD

12 cores · 32 GB RAM
500 GB NVMe SSD

All replication traffic travels over the private NIC (`eth1`). Public traffic is served via `eth0` through the floating VIP.

3. Hardware Specifications

Each box must be capable of running the full workload independently (survival mode).

	Box A (Primary)	Box B (Secondary)
CPU	16 cores	12 cores
RAM	32 GB	32 GB
Disk	500 GB NVMe SSD	500 GB NVMe SSD
NICs	2 × 1 Gbps	2 × 1 Gbps
Role	Active primary	Hot standby

4. Container Strategy

Docker Compose on each VM is the chosen approach.

- **Kubernetes** requires 3+ nodes for a resilient control plane — forcing it onto 2 nodes creates more problems than it solves at this scale.
- **Docker Swarm** is an option but adds orchestration complexity with minimal gain.
- **Docker Compose** provides container isolation, restart policies, health checks, and named networks — fully sufficient for this workload.

Keepalived runs on the host OS, not inside a container. It needs direct network interface access to manage the floating VIP; containerising it adds unnecessary complexity and risk.

5. Network Design

Interface	Box A	Box B	Purpose
<code>eth0</code>	<code>203.0.113.10</code>	<code>203.0.113.11</code>	Public traffic
<code>eth1</code>	<code>10.0.0.1</code>	<code>10.0.0.2</code>	Replication + cross-VM
Floating VIP	<code>203.0.113.100</code>	← same	Moves on failover

Firewall rules: ports `80` and `443` open on `eth0`. Ports `5432`, `6379`, `9000` bound to `eth1` only — never exposed publicly.

6. VMware Prerequisites

Confirm these settings before deployment:

Item	Required Setting
Port Group — Forged Transmits	Accept
Port Group — MAC Address Changes	Accept
VM NIC count	2 per VM (public + private)
Datastore	Separate datastores per VM if possible
vSphere HA	Disable for these two VMs (conflicts with Keepalived)
DRS Anti-Affinity Rule	Not required — already on separate physical hosts

7. Service Layout

Box A — Services

Service	Image	CPU	RAM	Exposed On
Nginx (LB + TLS)	<code>nginx:alpine</code>	1 core	512 MB	<code>eth0</code> 80/443
App instance 1	<code>ghcr.io/formbricks/formbricks</code>	2 cores	4 GB	internal 3001
App instance 2	<code>ghcr.io/formbricks/formbricks</code>	2 cores	4 GB	internal 3002
PostgreSQL (primary)	<code>pgvector/pgvector:pg17</code>	4 cores	10 GB	<code>eth1</code> 5432
Redis / Valkey (primary)	<code>valkey/valkey</code>	2 cores	2 GB	<code>eth1</code> 6379
MinIO (primary)	<code>minio/minio</code>	2 cores	4 GB	<code>eth1</code> 9000
Total		13 cores	24.5 GB	

Box B — Services

Service	Image	CPU	RAM	Exposed On
Nginx (LB + TLS)	<code>nginx:alpine</code>	1 core	512 MB	<code>eth0</code> 80/443
App instance 1	<code>ghcr.io/formbricks/formbricks</code>	2 cores	4 GB	internal 3001

PostgreSQL (hot standby)	pgvector/pgvector:pg17	4 cores	10 GB	eth1	5432
Redis / Valkey (replica)	valkey/valkey	2 cores	2 GB	eth1	6379
MinIO (replica)	minio/minio	2 cores	4 GB	eth1	9000
Total		11 cores	20.5 GB		

Box B has ~11.5 GB RAM headroom — enough to spin up a second app instance on failover.

8. Docker Compose — Box A

```
networks:
  frontend:
  backend:

volumes:
  postgres_data:
  redis_data:
  minio_data:
  uploads:

services:

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ./certs:/etc/nginx/certs:ro
    depends_on: [app-1, app-2]
    restart: unless-stopped
    networks: [frontend]

  app-1:
    image: ghcr.io/formbricks/formbricks:latest
    env_file: .env
    environment:
      PORT: 3001
    expose: ["3001"]
    volumes:
      - uploads:/home/nextjs/apps/web/uploads
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }
    restart: unless-stopped
    networks: [frontend, backend]

  app-2:
    image: ghcr.io/formbricks/formbricks:latest
    env_file: .env
    environment:
      PORT: 3002
    expose: ["3002"]
    volumes:
      - uploads:/home/nextjs/apps/web/uploads
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }
    restart: unless-stopped
    networks: [frontend, backend]
```

```

postgres:
  image: pgvector/pgvector:pg17
  environment:
    POSTGRES_USER: formbricks
    POSTGRES_PASSWORD: ${DB_PASSWORD}
    POSTGRES_DB: formbricks
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./postgres/primary.conf:/etc/postgresql/postgresql.conf:ro
    - ./postgres/pg_hba.conf:/etc/postgresql/pg_hba.conf:ro
  ports:
    - "10.0.0.1:5432:5432"
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U formbricks"]
    interval: 10s
    retries: 5
  restart: unless-stopped
  networks: [backend]

redis:
  image: valkey/valkey:latest
  command: valkey-server --requirepass ${REDIS_PASSWORD}
  volumes:
    - redis_data:/data
  ports:
    - "10.0.0.1:6379:6379"
  healthcheck:
    test: ["CMD", "valkey-cli", "-a", "${REDIS_PASSWORD}", "ping"]
    interval: 10s
    retries: 5
  restart: unless-stopped
  networks: [backend]

minio:
  image: minio/minio:latest
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: ${S3_ACCESS_KEY}
    MINIO_ROOT_PASSWORD: ${S3_SECRET_KEY}
  volumes:
    - minio_data:/data
  ports:
    - "10.0.0.1:9000:9000"
    - "10.0.0.1:9001:9001"
  restart: unless-stopped
  networks: [backend]

```

9. Docker Compose — Box B

```

networks:
  frontend:
  backend:

```

```

volumes:
  postgres_data:
  redis_data:
  minio_data:
  uploads:

services:

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ./certs:/etc/nginx/certs:ro
    depends_on: [app-1]
    restart: unless-stopped
    networks: [frontend]

  app-1:
    image: ghcr.io/formbricks/formbricks:latest
    env_file: .env
    environment:
      PORT: 3001
    expose: ["3001"]
    volumes:
      - uploads:/home/nextjs/apps/web/uploads
    restart: unless-stopped
    networks: [frontend, backend]

  postgres:
    image: pgvector/pgvector:pg17
    environment:
      POSTGRES_USER: formbricks
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: formbricks
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./postgres/replica.conf:/etc/postgresql/postgresql.conf:ro
    ports:
      - "10.0.0.2:5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U formbricks"]
      interval: 10s
      retries: 5
    restart: unless-stopped
    networks: [backend]

  redis:
    image: valkey/valkey:latest
    command: >
      valkey-server
      --requirepass ${REDIS_PASSWORD}
      --replicaof 10.0.0.1 6379
      --masterauth ${REDIS_PASSWORD}
    volumes:
      - redis_data:/data
    ports:

```

```
- "10.0.0.2:6379:6379"
restart: unless-stopped
networks: [backend]

minio:
  image: minio/minio:latest
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: ${S3_ACCESS_KEY}
    MINIO_ROOT_PASSWORD: ${S3_SECRET_KEY}
  volumes:
    - minio_data:/data
  ports:
    - "10.0.0.2:9000:9000"
  restart: unless-stopped
  networks: [backend]
```

10. Keepalived Configuration (Host OS)

Install on both VMs: `apt install keepalived`

Box A — `/etc/keepalived/keepalived.conf`

```
vrp_script chk_nginx {
    script "docker inspect --format '{{.State.Running}}' nginx | grep -q true"
    interval 3
    weight -20
}

vrp_instance VI_1 {
    state MASTER
    interface eth0
    virtual_router_id 51
    priority 100
    advert_int 1

    unicast_src_ip 203.0.113.10
    unicast_peer { 203.0.113.11 }

    authentication {
        auth_type PASS
        auth_pass <shared-secret>
    }

    virtual_ipaddress {
        203.0.113.100/24
    }

    track_script { chk_nginx }
}
```

Box B — identical config, except `state BACKUP` and `priority 90`.

Unicast mode is required because VMware vSwitch blocks VRRP multicast by default. If you prefer multicast, enable **Forged Transmits** and **MAC Address Changes** on the Port Group instead.

11. PostgreSQL Streaming Replication

Box A — `postgres/primary.conf`

```
wal_level = replica
max_wal_senders = 3
wal_keep_size = 256
synchronous_commit = off
max_connections = 200
shared_buffers = 2GB
work_mem = 64MB
```

Box A — postgres/pg_hba.conf (append)

```
host replication replicator 10.0.0.2/32 scram-sha-256
```

Bootstrap Box B replica (run once)

```
docker exec -it postgres bash
pg_basebackup -h 10.0.0.1 -U replicator \
  -D /var/lib/postgresql/data -P -R
```

The `-R` flag automatically writes `standby.signal` and `primary_conninfo`.

12. MinIO Bucket Replication

Run once after both MinIO instances are running:

```
# Configure aliases
mc alias set boxa http://10.0.0.1:9000 $S3_ACCESS_KEY $S3_SECRET_KEY
mc alias set boxb http://10.0.0.2:9000 $S3_ACCESS_KEY $S3_SECRET_KEY

# Create buckets and enable versioning (required for replication)
mc mb boxa/formbricks && mc mb boxb/formbricks
mc version enable boxa/formbricks
mc version enable boxb/formbricks

# Set up one-way replication: Box A → Box B
mc replicate add boxa/formbricks \
  --remote-bucket http://$S3_ACCESS_KEY:$S3_SECRET_KEY@10.0.0.2:9000/formbricks \
  --replicate "delete,delete-marker,existing-objects"
```

13. Nginx Load Balancer Config (Both Boxes)

```
upstream formbricks {
    least_conn;
    server app-1:3001;
    server app-2:3002;    # Remove on Box B (only has app-1)
    keepalive 32;
}

server {
    listen 80;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name your-domain.com;

    ssl_certificate      /etc/nginx/certs/fullchain.pem;
    ssl_certificate_key  /etc/nginx/certs/privkey.pem;

    client_max_body_size 25m;

    location / {
        proxy_pass http://formbricks;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

14. Formbricks Environment Variables

Deploy identical `.env` on both boxes. Update IP addresses during failover.

```
# Application
WEBAPP_URL=https://your-domain.com
NEXTAUTH_URL=https://your-domain.com
NEXTAUTH_SECRET=<openssl rand -hex 32>
ENCRYPTION_KEY=<exactly-32-chars>
CRON_SECRET=<openssl rand -hex 32>

# Database – update to 10.0.0.2 on Box A failure
DATABASE_URL=postgresql://formbricks:${DB_PASSWORD}@10.0.0.1:5432/formbricks?connection_limit=15

# Cache – update to 10.0.0.2 on Box A failure
REDIS_URL=redis://:${REDIS_PASSWORD}@10.0.0.1:6379

# Object storage – update to 10.0.0.2 on Box A failure
S3_ACCESS_KEY=<key>
S3_SECRET_KEY=<secret>
S3_BUCKET_NAME=formbricks
S3_ENDPOINT_URL=http://10.0.0.1:9000
S3_FORCE_PATH_STYLE=1

# Email
MAIL_FROM=noreply@your-domain.com
SMTP_HOST=<smtp-host>
SMTP_PORT=587
SMTP_USER=<user>
SMTP_PASSWORD=<pass>
SMTP_SECURE_ENABLED=0
SMTP_AUTHENTICATED=1

# Tuning
LOG_LEVEL=info
TELEMETRY_DISABLED=1
SESSION_MAX_AGE=3600
```

`connection_limit=15` per app instance × 3 instances = 45 total PostgreSQL connections — well within safe limits.

15. Failure Scenarios

Failure	User Impact	Recovery
---------	-------------	----------

Box B goes down	1 app instance lost, replicas stop	Zero action — Box A handles full load. Restart Box B when ready.
Box A goes down	VIP moves in ~3 sec	Promote PG + Redis on Box B, update <code>.env</code> . ~1–5 min total.
Box A network flap	Keepalived moves VIP automatically	Auto-recovers when network returns.
PostgreSQL only fails	App returns DB errors	Promote Box B replica.

16. Box A Failover Runbook

```
# Step 1 – VIP moves to Box B automatically via Keepalived (~3 sec)

# Step 2 – Promote PostgreSQL on Box B
docker exec postgres pg_ctl promote -D /var/lib/postgresql/data

# Step 3 – Promote Redis on Box B
docker exec redis valkey-cli -a $REDIS_PASSWORD REPLICAOF NO ONE

# Step 4 – Update .env on Box B to point to local services
sed -i 's/10.0.0.1/10.0.0.2/g' .env
docker compose up -d --force-recreate app-1

# Step 5 – Start second app instance for full capacity
docker compose up -d app-2

# Total expected downtime: ~1–5 minutes
```

Restoring Box A After Recovery

```
# 1. Start services on Box A
docker compose up -d postgres redis minio

# 2. Re-sync PostgreSQL as new replica from Box B (now primary)
pg_basebackup -h 10.0.0.2 -U replicator \
  -D /var/lib/postgresql/data -P -R

# 3. Re-enable Redis replication from Box B
docker exec redis valkey-cli -a $REDIS_PASSWORD REPLICAOF 10.0.0.2 6379

# 4. Return VIP to Box A – temporarily raise keepalived priority on Box A
# or simply restart keepalived (Box A has priority 100 > Box B's 90)
systemctl restart keepalived
```

17. Deployment Checklist

VMware / Network

- ☐ Both VMs on separate physical ESXi hosts (confirmed)
- ☐ Port Group: Forged Transmits → Accept
- ☐ Port Group: MAC Address Changes → Accept
- ☐ 2 NICs per VM configured (public + private)
- ☐ Separate datastores for Box A and Box B volumes
- ☐ vSphere HA disabled for both VMs
- ☐ Private NIC firewall open between 10.0.0.1 ↔ 10.0.0.2

Host OS (Both Boxes)

- ☐ Docker Engine 24+ and Docker Compose v2 installed
- ☐ Keepalived installed (`apt install keepalived`)
- ☐ Keepalived configured in unicast mode
- ☐ VIP confirmed floating correctly (test with `ip addr show eth0`)

Box A Services

- ☐ PostgreSQL primary running and healthy
- ☐ Redis primary running and healthy
- ☐ MinIO primary running, bucket created, versioning enabled
- ☐ Both app instances running and responding
- ☐ Nginx proxying to both app instances on 443

Box B Services

- ☐ PostgreSQL replica streaming from Box A (`pg_stat_replication` shows active slot)
- ☐ Redis replica replicating from Box A
- ☐ MinIO replica receiving bucket replication
- ☐ One app instance running and responding
- ☐ Nginx configured (single upstream)

Formbricks

- ☐ `ENCRYPTION_KEY` is exactly 32 characters
- ☐ `WEBAPP_URL` uses `https://`
- ☐ SMTP email sending tested
- ☐ File upload tested (MinIO)
- ☐ Survey creation and submission tested end-to-end

Failover Test (Pre-Production)

- ☐ Shut down Box A — confirm VIP moves to Box B within 5 seconds
- ☐ Run failover runbook — confirm app stays accessible
- ☐ Restore Box A — confirm replication resumes cleanly
- ☐ Re-run load test against failover configuration

18. Summary

Layer	Box A	Box B	Failover
VIP / Load Balancer	Nginx + Keepalived MASTER	Nginx + Keepalived BACKUP	Auto ~3 sec
Application	2 containers	1 container (+1 on failover)	Auto via VIP
Database	PostgreSQL primary	PostgreSQL hot standby	Manual promote ~1 min
Cache / Sessions	Redis primary	Redis replica	Manual promote ~30 sec
Object Storage	MinIO primary	MinIO replicated	Manual env update ~1 min
Container Runtime	Docker Compose	Docker Compose	Independent stacks

Total expected recovery time from Box A complete failure: ~1–5 minutes (dominated by the manual database promotion step). All other layers recover automatically or within seconds.